DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-2

ASSIGNMENT 2 – CO2

CO2 Coverage: N-grams Models of Syntax; Counting Words; Unsmoothed N-grams; Smoothing — Backoff, Deleted Interpolation; Entropy; English Word Classes; Tagsets for English; Part of Speech Tagging; Rule-Based Part of Speech Tagging; Stochastic Part of Speech Tagging; Transformation-Based Tagging

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: __06-11-2026_____

Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline (Total Marks – 50)

Duration: 1 week

Scenario — Speech Transcript Analytics for a Transport Enquiry Helpline

A state transport corporation runs a telephone enquiry helpline that takes about four thousand calls a day about bus and train timings, routes, fares, cancellations and refunds. The calls are transcribed automatically, and the speech recogniser does not return one transcript but several competing hypotheses for every utterance: "two tickets to Vellore", "to tickets to Vellore" and "two tickets to Velore" all come back with similar acoustic scores. The recogniser cannot choose between them, because the decision is linguistic, not acoustic. Once a transcript is chosen it must be tagged before the origin, destination, date and fare can be pulled out of it, since "book", "return" and "stand" are each a noun in one call and a verb in the next.

You are the language-engineering team. You must build the n-gram language model that rescores the recogniser's hypotheses, measure it honestly with entropy and perplexity on held out data, design a tagset for the domain, and build and compare three part-of-speech taggers — rule-based, stochastic and transformation-based. Every claim you make about one model being better than another must rest on a measurement you made yourself.

Planning data and constraints

- Corpus: at least 20,000 transcribed utterances totalling at least 150,000 word tokens (real or synthetic, but documented), split 80 / 10 / 10 into training, development and test sets with no utterance appearing in more than one split.
- Recogniser output: at least 200 utterances for which five competing hypotheses and the correct reference transcript are available, so that rescoring can be evaluated.

- Tagset (design and document your own): between 20 and 30 tags covering the open and closed word classes of the domain, with an explicit mapping to the Penn Treebank tagset.
- Annotated data: at least 3,000 tokens hand-tagged with your tagset, of which at least 500 tokens are tagged independently by two people so that agreement can be reported.
- Evaluation: a held-out set that is never used for training or for tuning any weight, and a fixed random seed for every experiment so that results can be reproduced.
- Any manual effort (transcription, tagset design, annotation, rule writing, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The helpline is extended to metro and intercity rail services and begins to receive code-mixed speech, so the vocabulary grows sharply and more than fifteen per cent of the words in new calls are never seen in training. Re-evaluate your language model and your taggers and decide whether to retain, modify or redesign them.

TASK 1 – Counting Words and Building the Vocabulary 3 Marks

Guiding Enquiry

What exactly is being counted when we count the words of a corpus, and how do those decisions change every model built on top of them?

Investigate and interpret:

- Distinguish type, token, lemma and word form, and report all four counts for your corpus; state how you treated case, punctuation, numerals, disfluencies and the utterance boundary markers <s> and </s>.
- Justify every tokenisation decision that changes the counts — whether "Vellore" and "vellore" are one type, whether "6:30" is one token or three — and show the effect of two different decisions on the vocabulary size.
- Plot the frequency of each word against its rank, state Zipf's law, and check how closely your corpus follows it; identify the words that deviate most and explain why the domain causes the deviation.
- Build the vocabulary with an unknown-word threshold, mapping rare words to <UNK>; report the vocabulary size at thresholds of 1, 2, 3 and 5 and the proportion of tokens each threshold sends to <UNK>.
- Measure the out-of-vocabulary rate of the development set against the training vocabulary and explain why this single number limits everything that follows.

Evidence Expected

Documented counting decisions, a Zipf plot for the corpus, and vocabulary and OOV figures at several thresholds.

1. Aim

To analyse the transport enquiry corpus, count words and vocabulary, study Zipf's law, handle rare words using the UNK token, and calculate the out-of-vocabulary (OOV) rate.

2. Basic terminology

Term	Meaning	Example
Token	Every occurrence of a word	bus bus train = 3 tokens
Type	A unique word	bus, train = 2 types
Lemma	Dictionary/base form	booking → book
Word form	Actual grammatical form	book, books, booked, booking

3. Tokenisation decisions

Use the following consistent rules:

1. Convert all words to lowercase, so Vellore and vellore count as one type.
2. Remove punctuation that does not carry meaning.
3. Keep numerals and times as meaningful tokens.
4. Treat 6:30 as one token.
5. Preserve disfluencies such as um and uh because they occur in real speech.
6. Add <s> and </s> to mark the beginning and end of each utterance. Do not count these as ordinary lexical words.

Two tokenisation approaches:

- Approach 1: 6:30 is one token.
- Approach 2: 6:30 is split into 6, :, and 30.

The second approach increases the number of tokens and may increase vocabulary size. Exact changes must be calculated using the corpus.

4. Python program for word counting

import refrom collections import Counterwith open("corpus.txt", "r", encoding="utf-8") as file: text = file.read().lower()tokens = re.findall(r"<s>|</s>|[a-z]+(?:'[a-z]+)?|\d+(?::\d+)?", text)word_count = len(tokens)vocabulary = set(tokens)type_count = len(vocabulary)print("Total tokens:", word_count)print("Total types:", type_count)frequency = Counter(tokens)print("Most common words:")print(frequency.most_common(10))

5. Zipf's law

Zipf's law states that the frequency of a word is approximately inversely proportional to its

rank.

$$ f(r)\propto \frac{1}{r} $$

Where:

- $f(r)$ = frequency of a word
- r = rank of that word

For example, the most frequent words may be the, to, bus, and is. A log-log plot of word rank against frequency should approximately form a straight descending line.

6. Vocabulary and UNK threshold

Rare words are replaced by <UNK> so that the model can handle unseen words.

Threshold	Vocabulary rule
1	Keep words occurring at least once
2	Keep words occurring at least twice
3	Keep words occurring at least three times
5	Keep words occurring at least five times

Python:

from collections import Countercounts = Counter(tokens)for threshold in [1, 2, 3, 5]: vocab = {w for w, c in counts.items() if c >= threshold} rare = sum(c for w, c in counts.items() if c < threshold) percentage = rare / len(tokens) * 100 print("Threshold:", threshold) print("Vocabulary size:", len(vocab)) print("UNK percentage:", round(percentage, 2))

7. OOV rate

OOV means words in the development set that are not present in the training vocabulary.

$$ OOV\ Rate= \frac{\text{Unseen development tokens}} {\text{Total development tokens}}\times100 $$

A high OOV rate makes language modelling and tagging more difficult because the model has not learned those words.

8. Result

The corpus is tokenised, and its word counts, vocabulary size, Zipf distribution, UNK

vocabulary and OOV rate can be calculated using the above program. The actual numerical results must be filled in after running it on the prepared corpus.

TASK 2 – Unsmoothed N-gram Models 4 Marks

Guiding Enquiry

How is the probability of a word sequence estimated from counts alone, and where does that estimate break down?

Investigate and interpret:

- State the chain rule for the probability of a word sequence and derive the Markov assumption that turns it into a bigram and then a trigram model, saying precisely what is being assumed away at each step.
- Give the maximum likelihood estimator for unigram, bigram and trigram probabilities, and build all three models from your training split.
- Compute by hand the unsmoothed probability of at least three helpline utterances, showing every factor; choose one utterance that is perfectly reasonable English but receives probability zero, and identify the exact bigram responsible.
- Report the sparsity of your models: the proportion of possible bigrams and trigrams actually observed, and the proportion of development-set bigrams and trigrams never seen in training.
- Use the models as generators: sample ten utterances from the unigram, bigram and trigram models and comment on what the increase in order buys and what it costs.

Evidence Expected

Working unsmoothed models, hand-computed sentence probabilities including a zero, and quantified sparsity figures.

Aim

To build unigram, bigram and trigram language models using Maximum Likelihood Estimation (MLE) and understand the limitations of unsmoothed models.

1. Chain Rule

The probability of a sentence is calculated using the chain rule:

\[P(w_1,w_2,\ldots,w_n)=\prod_{i=1}^{n}P(w_i|w_1,\ldots,w_{i-1}) \]

This means that the probability of each word depends on all the words that came before it.

Unigram model

Assumes each word is independent:

\[P(w_1,\ldots,w_n)\approx\prod_iP(w_i) \]

Bigram model

Assumes a word depends only on the previous word:

\[P(w_1,\ldots,w_n)\approx\prod_iP(w_i|w_{i-1}) \]

Trigram model

Assumes a word depends only on the previous two words:

\[P(w_1,\ldots,w_n)\approx\prod_iP(w_i|w_{i-2},w_{i-1}) \]

The unigram model ignores word order. The bigram model ignores words before the immediately previous word. The trigram model ignores all context beyond the previous two words.

2. Maximum Likelihood Estimation

Let \(C(w)\) represent the number of occurrences of a word.

Unigram:

\[P(w)=\frac{C(w)}{N} \]

Bigram:

\[P(w_i|w_{i-1})= \frac{C(w_{i-1},w_i)}{C(w_{i-1})} \]

Trigram:

\[P(w_i|w_{i-2},w_{i-1})= \frac{C(w_{i-2},w_{i-1},w_i)} {C(w_{i-2},w_{i-1})} \]

3. Python Implementation

from collections import Countersentences = [["<s>", "two", "tickets", "to", "vellore", "</s>"], ["<s>", "book", "a", "bus", "ticket", "</s>"], ["<s>", "train", "to", "chennai", "</s>"]]unigrams = Counter()bigrams = Counter()trigrams = Counter()for s in sentences: unigrams.update(s) bigrams.update(zip(s[:-1], s[1:])) trigrams.update(zip(s[:-2], s[1:-1], s[2:]))total = sum(unigrams.values())def unigram_prob(word): return unigrams[word] / totaldef bigram_prob(w1, w2): return bigrams[(w1, w2)] / unigrams[w1]def trigram_prob(w1, w2, w3): return trigrams[(w1, w2, w3)] / bigrams[(w1, w2)]print("Unigram:", unigram_prob("to"))print("Bigram:", bigram_prob("to", "vellore"))print("Trigram:", trigram_prob("tickets", "to", "vellore"))

This small corpus demonstrates the calculation. Replace it with the actual training corpus for the assignment.

4. Hand Calculation

Suppose the training corpus contains:

Count	Value
Total word tokens	100
Count of to	10
Count of to vellore	4
Count of tickets to	3
Count of tickets to vellore	2
Count of tickets to as a bigram	3

Then:

$$ P(to)=\frac{10}{100}=0.10 $$

$$ P(vellore|to)=\frac{4}{10}=0.40 $$

$$ P(vellore|tickets,to)=\frac{2}{3}=0.667 $$

For the sentence tickets to vellore, ignoring boundary factors:

$$ P= P(tickets)\times P(to|tickets)\times P(vellore|tickets,to) $$

Use the actual corpus counts to complete the numerical sentence probability.

5. Zero Probability Problem

Suppose the phrase tickets to madurai is grammatically correct, but the bigram to madurai never appears in training.

Then:

$$ P(madurai|to)=\frac{0}{C(to)}=0 $$

Therefore, the entire sentence probability becomes zero.

This is a limitation of unsmoothed models: an unseen phrase is treated as impossible even when it is valid English.

6. Sparsity

Bigram sparsity:

$$ S_2=1-\frac{\text{Observed bigrams}}{\text{Possible bigrams}} $$

Trigram sparsity:

\[S_3=1-\frac{\text{Observed trigrams}}{\text{Possible trigrams}} \]

Development-set unseen bigram rate:

\[U_2=\frac{\text{Unseen development bigrams}}{\text{Total development bigrams}}\times100 \]

The same formula applies to trigrams.

7. Sentence Generation

- Unigram: selects words without considering order, so sentences may be meaningless.
- Bigram: considers the previous word, producing somewhat more meaningful sequences.
- Trigram: considers two previous words, usually producing more natural local word sequences.
- Trigram models require more memory and suffer from greater sparsity.

Result

Unigram, bigram and trigram models can be constructed using MLE. Unsmoothed models assign zero probability to unseen word sequences, which motivates smoothing techniques.

TASK 3 – Smoothing and Discounting 5 Marks

Guiding Enquiry

How can probability mass be moved from what was seen to what was not, without destroying what the counts tell us?

Investigate and interpret:

- Explain why a zero probability is a modelling failure rather than a fact about the language, and state the condition any smoothing method must satisfy for the distribution to remain valid.
- Implement add-one (Laplace) and add-k smoothing; for a bigram with a left context of your choice, tabulate the original counts, the effective (discounted) counts and the resulting probabilities, and show exactly how much mass was taken from the seen bigrams.
- Explain why add-one smoothing discounts so severely when the vocabulary is large, with the arithmetic for one context from your own corpus.
- Implement Good–Turing discounting: build the frequency-of-frequencies table N_r, compute the revised counts c*, estimate the probability of all unseen bigrams, and explain how the high-frequency tail is handled.

- Compare the methods on the development set by perplexity and by how reasonable the estimate for a specific unseen bigram looks, and state which you would deploy and why.

Evidence Expected

Correctly implemented smoothing methods with discount tables, a Good–Turing N_r table, and a measured comparison.

Aim

To implement smoothing techniques that assign non-zero probabilities to unseen word sequences.

1. Why Smoothing Is Needed

A language model trained on a limited corpus cannot observe every possible word combination.

For example, bus to vellore may occur in training, but train to vellore may not.

Without smoothing:

\[P(\text{unseen sequence})=0 \]

This causes a valid sentence to receive zero probability.

A valid probability distribution must satisfy:

\[\sum_wP(w|h)=1 \]

where \(h\) is the previous context.

2. Add-One (Laplace) Smoothing

Add one to every possible word count.

\[P(w|h)=\frac{C(h,w)+1}{C(h)+V} \]

Here:

- \(C(h,w)\) = bigram count.
- \(C(h)\) = context count.
- \(V\) = vocabulary size.

Example: Suppose \(C(to)=10\), \(C(to, vellore)=4\), and \(V=20\).

\[P(vellore|to)=\frac{4+1}{10+20} =\frac{5}{30}=0.1667 \]

An unseen word receives:

\[P(unseen|to)=\frac{0+1}{30}=0.0333 \]

Add-one smoothing can reduce probabilities of frequently observed words too much when the vocabulary is large.

3. Add-k Smoothing

$$P(w|h)=\frac{C(h,w)+k}{C(h)+kV}$$

For example, when $k=0.1$:

$$P(vellore|to)=\frac{4+0.1}{10+(0.1)(20)}=\frac{4.1}{12}=0.3417$$

Add-k usually applies less smoothing than add-one when k is small.

4. Python Code

```
def add_k_probability(count, context_count, vocab_size, k): return (count + k) / (context_count + k * vocab_size)print(add_k_probability(4, 10, 20, 1))print(add_k_probability(4, 10, 20, 0.1))
```

5. Good-Turing Discounting

Good-Turing estimates adjusted counts using the frequency of frequencies.

$$N_r=\text{Number of n-grams occurring exactly r times}$$

Adjusted count:

$$c^*=(c+1)\frac{N_{c+1}}{N_c}$$

Probability of unseen n-grams:

$$P_0=\frac{N_1}{N}$$

Here, N_1 is the number of n-grams seen exactly once.

Good-Turing assigns some probability mass to unseen n-grams by reducing the counts of observed n-grams.

6. Good-Turing Python Example

```
from collections import Countercounts = Counter({ "to vellore": 4, "to chennai": 2, "to madurai": 1, "bus timing": 1})frequency = Counter(counts.values())N = sum(counts.values())for gram, count in counts.items(): next_count = frequency.get(count + 1, 0) current_count = frequency.get(count, 0) if current_count and next_count: adjusted = (count + 1) * next_count / current_count print(gram, adjusted) else: print(gram, count)
```

This is a basic illustration. A production Good-Turing implementation needs a suitable strategy for missing frequency classes and the high-frequency tail.

7. Comparison

Method	Advantage	Limitation
Add-one	Very simple	Excessive discounting
Add-k	Adjustable smoothing	Requires selecting k
Good-Turing	Uses frequency-of-frequencies	Needs careful handling of sparse frequency classes

Result

Smoothing prevents unseen n-grams from receiving zero probability. The final method should be selected by comparing development-set perplexity and the quality of probabilities assigned to unseen phrases.

TASK 4 – Backoff and Deleted Interpolation 4 Marks

Guiding Enquiry

When the trigram has never been seen, how should the model fall back on the shorter contexts it does know?

Investigate and interpret:

- Define backoff and interpolation and state the essential difference: backoff consults a shorter context only when the longer one fails, interpolation always combines all orders.
- Implement Katz backoff with Good–Turing discounting, explaining the role of the normalising weight α and why the model is not a valid distribution without it; also implement stupid backoff and say why its output is a score and not a probability.
- Implement simple linear interpolation, then deleted interpolation: hold out part of the training data, estimate the λ weights on it, and report the weights you obtain for at least three context-count bands.
- Explain why the λ weights should depend on the count of the context, and show with one worked example from your corpus how a low-count context shifts weight onto the lower orders.
- Compare backoff and interpolation on the held-out set by perplexity, and hand-trace one utterance through both so that the difference in treatment of a single unseen trigram is visible.

Evidence Expected

Working backoff and interpolation models, estimated λ weights with the held-out procedure described, and a traced comparison.

Aim

To implement backoff and interpolation methods for estimating probabilities when longer n-gram contexts are unavailable.

1. Backoff

Backoff uses a lower-order model only when the higher-order model cannot provide a reliable probability.

For example:

1. Try trigram probability.
2. If the trigram is unseen, use the bigram.
3. If the bigram is also unseen, use the unigram.

Katz Backoff:

$$P_{Katz}(w|h)= \begin{cases} P^*(w|h), & C(h,w)>0\\ \alpha(h)P_{Katz}(w|h'), & C(h,w)=0 \end{cases}$$

Here:

- P^* is the discounted probability.
- h' is the shorter context.
- $\alpha(h)$ is the normalising weight.

The value of α ensures that the probabilities sum to one after backing off.

2. Stupid Backoff

$$S(w|h)= \begin{cases} \frac{C(h,w)}{C(h)}, & \text{if observed}\\ 0.4S(w|h'), & \text{otherwise} \end{cases}$$

Stupid Backoff is easy to implement, but its output is a score, not a properly normalised probability.

3. Linear Interpolation

Interpolation combines all model orders:

$$P(w_i)= \lambda_1P(w_i)+ \lambda_2P(w_i|w_{i-1})+ \lambda_3P(w_i|w_{i-2},w_{i-1})$$

The weights satisfy:

$$ \lambda_1+\lambda_2+\lambda_3=1 $$

For example:

$$ \lambda_1=0.2,\quad \lambda_2=0.3,\quad \lambda_3=0.5 $$

If the three probabilities are 0.1, 0.2 and 0.4:

$$ P=0.2(0.1)+0.3(0.2)+0.5(0.4) $$

$$ P=0.28 $$

4. Python Code

def interpolate(p1, p2, p3, l1, l2, l3): return l1*p1 + l2*p2 + l3*p3p = interpolate(0.1, 0.2, 0.4, 0.2, 0.3, 0.5)print("Interpolated probability:", p)

5. Deleted Interpolation

Deleted interpolation estimates weights using held-out portions of training data.

Procedure:

1. Divide training data into parts.
2. Temporarily remove one part.
3. Estimate unigram, bigram and trigram probabilities using the remaining data.
4. For each held-out event, identify which model gives the highest probability.
5. Add the event's count to that model's weight.
6. Repeat and normalise the weights.

$$ \lambda_i=\frac{\text{Weight assigned to model i}}{\text{Total assigned weight}} $$

The resulting weights must be calculated from the actual training corpus. Do not assume the example weights above are experimental results.

6. Why Weights Depend on Context Count

A high-count trigram provides stronger evidence and can receive more weight.

A low-count trigram is less reliable, so the model should depend more on bigram and unigram probabilities.

Result

Backoff uses shorter contexts when longer contexts fail. Interpolation combines all model

orders. Their performance must be compared using held-out perplexity.

TASK 5 – Entropy, Cross-Entropy and Perplexity 4 Marks

Guiding Enquiry

How is a language model measured, and what does the number actually

mean? **Investigate and interpret:**

- Define entropy, cross-entropy and perplexity, state the relation between them, and explain in plain words what a perplexity of 90 tells an engineer about the model.
- Compute the entropy of at least three distributions from your own model by hand — including one that is nearly uniform and one that is sharply peaked — and explain what the difference means for prediction.
- Compute the cross-entropy and perplexity of the held-out set under each model built in Tasks 2 to 4, and present the results in one comparable table; state the per-word and per-utterance figures separately.
- Explain why perplexity may only be compared between models that share a vocabulary and the same treatment of <UNK>, and demonstrate the distortion with a deliberately unfair comparison of your own.
- Use your 200 recogniser hypothesis sets: rescore each set with your best model, report the word error rate before and after rescoring, and state whether perplexity improvements translated into error reductions.

Evidence Expected

Correct entropy calculations, a comparable perplexity table, and a rescoring experiment relating perplexity to word error rate.

Aim

To evaluate language models using entropy, cross-entropy and perplexity.

1. Entropy

Entropy measures uncertainty in a probability distribution.

\[H(X)=-\sum_xP(x)\log_2P(x) \]

A high entropy indicates greater uncertainty. A low entropy indicates that one or a few outcomes are more likely.

Example – nearly uniform distribution:

\[P=(0.5,0.5) \]

\[H=-0.5\log_2(0.5)-0.5\log_2(0.5)=1 \]

Example – sharply peaked distribution:

\[P=(0.9,0.1) \]

\[H=-0.9\log_2(0.9)-0.1\log_2(0.1) \approx0.469 \]

The sharply peaked distribution has lower uncertainty.

2. Cross-Entropy

Cross-entropy measures how well a model predicts actual test data.

\[H(P,Q)=-\frac{1}{N}\sum_{i=1}^{N}\log_2Q(w_i|context_i) \]

A lower cross-entropy indicates better prediction.

3. Perplexity

\[PP=2^{H(P,Q)} \]

A perplexity of 90 means that, roughly speaking, the model is as uncertain as choosing among 90 equally likely alternatives at each prediction point. It does not mean that the model literally has only 90 possible words.

4. Python Code

import mathdef perplexity(probabilities): total = 0 for p in probabilities: if p <= 0: return float("inf") total += math.log2(p) cross_entropy = -total / len(probabilities) return 2 ** cross_entropyprobs = [0.5, 0.4, 0.8, 0.6]print("Perplexity:", perplexity(probs))

Use smoothed probabilities for the test set. An unsmoothed model may produce infinite perplexity if any test event has probability zero.

5. Model Comparison Table

Fill this table with values calculated on the same held-out test set.

Model	**Cross-Entropy**	**Perplexity**
Unigram	Calculate	Calculate
Bigram	Calculate	Calculate
Trigram	Calculate	Calculate
Add-one	Calculate	Calculate
Add-k	Calculate	Calculate
Good-Turing	Calculate	Calculate
Katz Backoff	Calculate	Calculate
Interpolation	Calculate	Calculate

Per-word cross-entropy is the average negative log probability per predicted word. Per-utterance cross-entropy is the sum of the negative log probabilities for that utterance; report the average across utterances separately.

6. Why Vocabulary Must Be the Same

Perplexity comparisons are fair only when models use the same vocabulary and the same unknown-word (<UNK>) treatment.

If one model maps many rare words to <UNK> and another keeps them separate, their prediction tasks differ. Their perplexities cannot be compared directly.

7. Word Error Rate

\[WER=\frac{S+D+I}{N}\times100 \]

Where:

- S = substitutions
- D = deletions
- I = insertions
- N = words in the reference transcript

Rescore the five hypotheses for each of the 200 utterances using the language model. Compare the selected transcript's WER against the recogniser's original selection.

Result

Entropy measures uncertainty, cross-entropy measures predictive quality and perplexity expresses cross-entropy on an exponential scale. The best model should be evaluated using both perplexity and actual recognition WER.

TASK 6 – English Word Classes and Tagset Design 4 Marks

Guiding Enquiry

What are the word classes of English, and how many distinctions should a tagset for this helpline actually make?

Investigate and interpret:

- Distinguish open and closed word classes, list the members of the closed classes that occur in your corpus, and explain why the closed classes matter disproportionately for tagging.
- Define the major open classes with the morphological and distributional tests used to identify them, and apply those tests to at least five ambiguous words from your corpus.
- Design your tagset of 20 to 30 tags, justify each distinction you chose to make and at least three you chose not to make, and give the full mapping to the Penn Treebank tagset.
- Annotate your 3,000-token sample, report the tag frequency distribution, and give the inter-annotator agreement on the doubly annotated portion as a raw percentage and as Cohen's kappa.
- Measure the ambiguity your tagset leaves: the proportion of word types and of word tokens with more than one possible tag, and the average number of tags per ambiguous token.

Evidence Expected

A justified tagset with a Penn Treebank mapping, an annotated sample with agreement

figures, and a measurement of residual ambiguity.

Aim

To identify English word classes and design a POS tagset suitable for transport enquiry transcripts.

1. Open and Closed Word Classes

Open classes	**Examples**
Nouns	bus, train, ticket, station
Verbs	book, cancel, travel, return
Adjectives	available, delayed, fast
Adverbs	today, tomorrow, quickly

Open classes accept new words.

Closed classes	**Examples**
Determiners	a, the, this
Pronouns	I, you, we
Prepositions	to, from, at
Conjunctions	and, but, or
Auxiliary verbs	is, are, can
Particles	up, off

Closed classes are important because they occur frequently and help determine grammatical relationships.

2. Morphological and Distributional Tests

- Noun: can occur after the; example, the ticket.
- Verb: can take tense forms; example, book, booked.
- Adjective: can occur before a noun; example, available ticket.
- Adverb: can modify a verb; example, travel quickly.

Ambiguous examples:

Word	Noun use	Verb use
book	Read a book	Book a ticket
return	A return ticket	Return tomorrow
stand	Bus stand	Please stand
train	The train	Train the staff
call	A call	Call the helpline

3. Proposed Domain Tagset

A 24-tag design is shown below. This is a proposed tagset; it should be checked against the actual corpus.

Tag	Meaning	Penn Treebank mapping
NNP	Proper noun	NNP
NN	Common singular noun	NN
NNS	Plural noun	NNS
VB	Base verb	VB
VBD	Past verb	VBD
VBG	Gerund/present participle	VBG
VBN	Past participle	VBN
VBZ	3rd-person singular verb	VBZ
VBP	Non-3rd-person present verb	VBP
JJ	Adjective	JJ
RB	Adverb	RB
PRP	Personal pronoun	PRP

PRP$	Possessive pronoun	PRP$
DT	Determiner	DT
IN	Preposition/subordinating conjunction	IN
CC	Coordinating conjunction	CC
MD	Modal	MD
CD	Number	CD
TO	Infinitive marker	TO
RP	Particle	RP
UH	Interjection	UH
WRB	Wh-adverb	WRB
WP	Wh-pronoun	WP
SYM	Special symbol/domain code	SYM

Distinctions not made:

1. Separate tags for different types of proper nouns are not used.
2. Separate tags for every kind of adjective are not used.
3. Different preposition subtypes are grouped under IN.

These decisions keep the tagset manageable.

4. Annotation and Agreement

Annotate at least 3,000 tokens. Have two annotators independently label at least 500 of those tokens.

Raw agreement:

$$Agreement=\frac{\text{Number of matching tags}}{\text{Total double-tagged tokens}}\times100$$

Cohen's kappa:

$$\kappa=\frac{P_o-P_e}{1-P_e}$$

Where \(P_o\) is observed agreement and \(P_e\) is expected agreement by chance.

5. Ambiguity

A word is ambiguous if it can receive more than one POS tag.

\[\text{Ambiguous token rate}= \frac{\text{Ambiguous tokens}}{\text{Total tokens}}\times100 \]

\[\text{Average tags per ambiguous token}= \frac{\text{Total candidate tags for ambiguous tokens}} {\text{Number of ambiguous tokens}} \]

Result

The proposed 24-tag system covers the major English word classes and transport-domain vocabulary. Actual tag frequencies, agreement and ambiguity must be calculated from the annotated sample.

TASK 7 – Rule-Based Part-of-Speech Tagging 4 Marks

Guiding Enquiry

How far can hand-written rules take a tagger, and what makes writing them hard? **Investigate and interpret:**

- Describe the two-stage architecture of a rule-based tagger: a dictionary that assigns every possible tag, then constraints that remove the impossible ones.
- Build the tag dictionary from your annotated data and report how many words receive one, two, three or more candidate tags.
- Write at least fifteen disambiguation rules in an explicit condition-action notation, each stating the context it inspects; include rules that use the preceding tag, the following word and the word's own suffix.
- Evaluate the tagger on the held-out set, report overall accuracy and accuracy on ambiguous tokens only, and explain why the second number is the one that matters.
- Analyse the failures: give the five commonest error types, show one case where two of your rules conflict, and explain how rule ordering resolved it and what that costs in maintainability.

Evidence Expected

A documented rule set with a tag dictionary, measured accuracy on ambiguous tokens, and a rule-conflict analysis.

Aim

To develop a rule-based POS tagger using a dictionary and grammatical rules.

1. Two-Stage Architecture

Stage 1 – Dictionary

Assigns all possible tags to each word.

Example:

Word	Possible tags
book	NN, VB
return	NN, VB
train	NN, VB
stand	NN, VB

Stage 2 – Disambiguation

Uses context rules to remove unlikely tags.

Example:

Please book a ticket.

The word book is a verb because it follows Please.

2. Fifteen Disambiguation Rules

Notation: IF condition THEN assign tag.

No.	Condition	Action
1	Word follows the	Assign NN/NNS
2	Word follows a or an	Assign NN
3	Word follows Please	Assign VB
4	Word follows can	Assign VB
5	Word follows will	Assign VB
6	Word ends in -ed	Consider VBD/VBN
7	Word ends in -ing	Consider VBG
8	Word ends in -ly	Consider RB
9	Word follows a possessive determiner	Assign NN
10	Word follows to and is a verb form	Assign VB
11	Word is a known station name	Assign NNP
12	Word follows from	Consider NNP/NN
13	Word follows very	Assign JJ/RB according to context
14	Word follows a subject pronoun	Consider VBP/VBZ

15	Word is a known number	Assign CD

These are candidate rules. The actual dictionary and rules should be validated against the corpus.

3. Python Example

dictionary = { "book": ["NN", "VB"], "ticket": ["NN"], "train": ["NN", "VB"], "please": ["UH"], "can": ["MD"]}def tag_sentence(words): tags = [] for i, word in enumerate(words): options = dictionary.get(word.lower(), ["NN"]) if i > 0 and words[i-1].lower() in ["please", "can"]: tags.append("VB") else: tags.append(options[0]) return list(zip(words, tags))print(tag_sentence(["Please", "book", "ticket"]))

4. Evaluation

\[Accuracy=\frac{\text{Correctly tagged tokens}}{\text{Total test tokens}}\times100 \]

Calculate accuracy separately for:

- All tokens.
- Ambiguous tokens only.

Ambiguous-token accuracy is important because these are the words for which the tagger must make a real choice.

5. Common Errors

1. A noun incorrectly tagged as a verb.
2. A verb incorrectly tagged as a noun.
3. Unknown station names incorrectly tagged as common nouns.
4. Incorrect handling of abbreviations.
5. Incorrect handling of disfluencies and incomplete sentences.

Rule conflict example:

- Rule 1: A word after the is a noun.
- Rule 6: A word ending in -ed may be a past verb.

In the delayed train, the first rule identifies delayed as an adjective or participle in context. Rules should be ordered and refined to resolve conflicts.

Result

A rule-based tagger can use word dictionaries and grammatical context to assign POS tags. Its accuracy depends on the quality of the dictionary and the correctness of its rules.

TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm 5

Marks Guiding Enquiry

How does a probabilistic tagger choose a whole tag sequence rather than one tag at a time? **Investigate and interpret:**

- Derive the HMM tagging equation from Bayes' rule, state the two independence assumptions it rests on, and identify the transition and emission terms.
- Estimate the transition and emission probabilities from your annotated data, state how you smoothed them, and show that each transition row sums to one.
- Hand-trace the Viterbi algorithm on one ambiguous helpline utterance of at least five words: fill every trellis cell, keep a back-pointer for each, recover the best tag sequence and give its probability.
- Compare Viterbi against the most-frequent-tag baseline and against scoring every tag sequence exhaustively; report the accuracy of each and the number of operations the three require for your sentence.
- Explain how unknown words are tagged by an HMM — suffix evidence, capitalisation, an open-class prior — implement one such method, and report its accuracy on the out of-vocabulary tokens alone.

Evidence Expected

A correctly estimated HMM, a fully traced Viterbi trellis with back-pointers, and a measured treatment of unknown words.

Aim

To implement a Hidden Markov Model POS tagger and use the Viterbi algorithm to find the most probable tag sequence.

1. HMM Tagging Equation

Using Bayes' rule:

$$P(T|W)\propto P(W|T)P(T)$$

The most probable tag sequence is:

$$T^*=\arg\max_T P(T|W)$$

Under the HMM assumptions:

1. A tag depends only on the previous tag.
2. A word depends only on its current tag.

Therefore:

$$T^*=\arg\max_T \prod_iP(t_i|t_{i-1})P(w_i|t_i)$$

- Transition probability: $P(t_i|t_{i-1})$
- Emission probability: $P(w_i|t_i)$

2. Probability Estimation

Transition:

$$ P(t_i|t_{i-1})= \frac{C(t_{i-1},t_i)}{C(t_{i-1})} $$

Emission:

$$ P(w_i|t_i)= \frac{C(t_i,w_i)}{C(t_i)} $$

Apply smoothing to avoid zero probabilities.

3. Viterbi Algorithm

For each word and candidate tag:

$$ V_i(t)=\max_s[V_{i-1}(s)P(t|s)P(w_i|t)] $$

Store the previous tag that gives the maximum value as a back-pointer.

Illustrative example:

Sentence: Please book two tickets

Candidate tags:

- Please: UH
- book: NN or VB
- two: CD
- tickets: NNS

A likely sequence is:

UH VB CD NNS

The complete assignment requires a five-word-or-longer sentence and a trellis containing every candidate tag, every cell probability and each back-pointer. The values must come from the annotated corpus; they cannot be calculated reliably without its transition and emission counts.

4. Python HMM Example

```
import nltkfrom nltk.tag import hmmtrain_data = [ [("Please", "UH"), ("book", "VB"), ("two", "CD"), ("tickets", "NNS")]]trainer = hmm.HiddenMarkovModelTrainer()tagger = trainer.train_supervised(train_data)sentence = ["Please", "book", "two", "tickets"]print(tagger.tag(sentence))
```

This is a small demonstration. Train the model using the full annotated training set.

5. Comparison

Method	Description
Most-frequent-tag baseline	Assigns the most common tag to each word
Exhaustive search	Checks every possible tag sequence
Viterbi	Uses dynamic programming to find the best sequence efficiently

For \(n\) words and \(T\) possible tags:

- Exhaustive search may examine \(T^n\) sequences.
- Viterbi takes approximately \(O(nT^2)\) operations.

6. Unknown Words

Unknown words can be handled using:

- Suffix patterns.
- Capitalisation.
- Number detection.
- Open-class tag priors.
- A special <UNK> category.

Example:

- Vellore → likely NNP because it begins with a capital letter and may appear in a station gazetteer.
- delayed → likely VBN/JJ because of its suffix and context.
- 500 → CD.

Evaluate unknown-word accuracy separately:

\[Accuracy_{UNK}= \frac{\text{Correct unknown-word tags}} {\text{Total unknown-word tokens}}\times100 \]

Result

An HMM selects the most probable complete tag sequence by combining transition and emission probabilities. Viterbi makes this process efficient compared with exhaustive search.

TASK 9 – Transformation-Based (Brill) Tagging 4 Marks

Guiding Enquiry

Can a tagger learn its own correction rules, and are the rules it learns worth reading? **Investigate and interpret:**

- Describe transformation-based learning: the initial-state annotator, the templates, the greedy selection of the transformation that corrects the most errors, and the stopping condition.
  - Define at least six rule templates over the preceding tag, the following tag, the surrounding words and the word's own form, and justify the window you chose.
- Run the learner from a most-frequent-tag initial state and report the first ten transformations learnt in order, with the number of errors each one corrected and introduced.
- Explain why the learnt rules are readable in a way that HMM probabilities are not, and show one learnt rule that encodes a genuine linguistic generalisation and one that is an artefact of your training data.
- Measure accuracy against the number of transformations applied, plot the curve, and identify the point beyond which further rules overfit.

Evidence Expected

A working Brill learner with its learnt rule list, and an accuracy curve identifying the overfitting point.

Aim

To implement transformation-based learning, which improves initial POS tags through learned correction rules.

1. Working

Brill tagging follows these steps:

1. Assign initial tags using a simple baseline.
2. Compare predicted tags with correct training tags.
3. Generate possible transformation rules.
4. Select the rule that corrects the most errors.
5. Apply the rule.
6. Repeat until the improvement falls below a stopping threshold.

2. Six Rule Templates

No.	Template
1	Change tag A to B if previous tag is X
2	Change tag A to B if next tag is X

3	Change tag A to B if previous word is X
4	Change tag A to B if next word is X
5	Change tag A to B if current word ends in X
6	Change tag A to B if current word is X

A small context window is useful because it captures nearby grammatical relationships without creating too many rules.

3. Example Transformations

These are illustrative examples, not learned rules from the user's corpus.

Rule	Transformation	Reason
1	NN → VB after Please	Commands usually take verbs
2	NN → VB after can	Modal verbs are followed by base verbs
3	NN → NNP if word is in station list	Station names are proper nouns
4	VB → NN after the	Determiner often precedes a noun
5	NN → RB for suitable -ly words	Many -ly words are adverbs
6	NN → VBG for suitable -ing words	Context may indicate a gerund

The assignment requires the first ten transformations, in learned order, with errors corrected and introduced. Those results must be obtained by running the learner on annotated data.

4. Python Demonstration

```
import nltktrain = [  [("Please", "UH"), ("book", "VB"),  ("a", "DT"), ("ticket", "NN")]]initial = nltk.DefaultTagger("NN")templates = nltk.brill.brill24()trainer = nltk.BrillTaggerTrainer(initial, templates)tagger = trainer.train(train, max_rules=10)print(tagger.tag(["Please", "book", "a", "ticket"]))
```

The code illustrates the Brill training approach. For a proper evaluation, use the full annotated corpus and a separate development set.

5. Accuracy and Overfitting

$$ Accuracy=\frac{\text{Correct tags}}{\text{Total tags}}\times100 $$

Plot:

- X-axis: Number of transformations.
- Y-axis: Training and development accuracy.

If training accuracy continues to increase while development accuracy decreases, the rules are overfitting.

Result

Brill tagging learns readable correction rules from annotated examples. It can capture useful linguistic patterns, but too many rules may overfit the training data.

TASK 10 – Implement and Test the Models 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by

hand? **Investigate and interpret:**

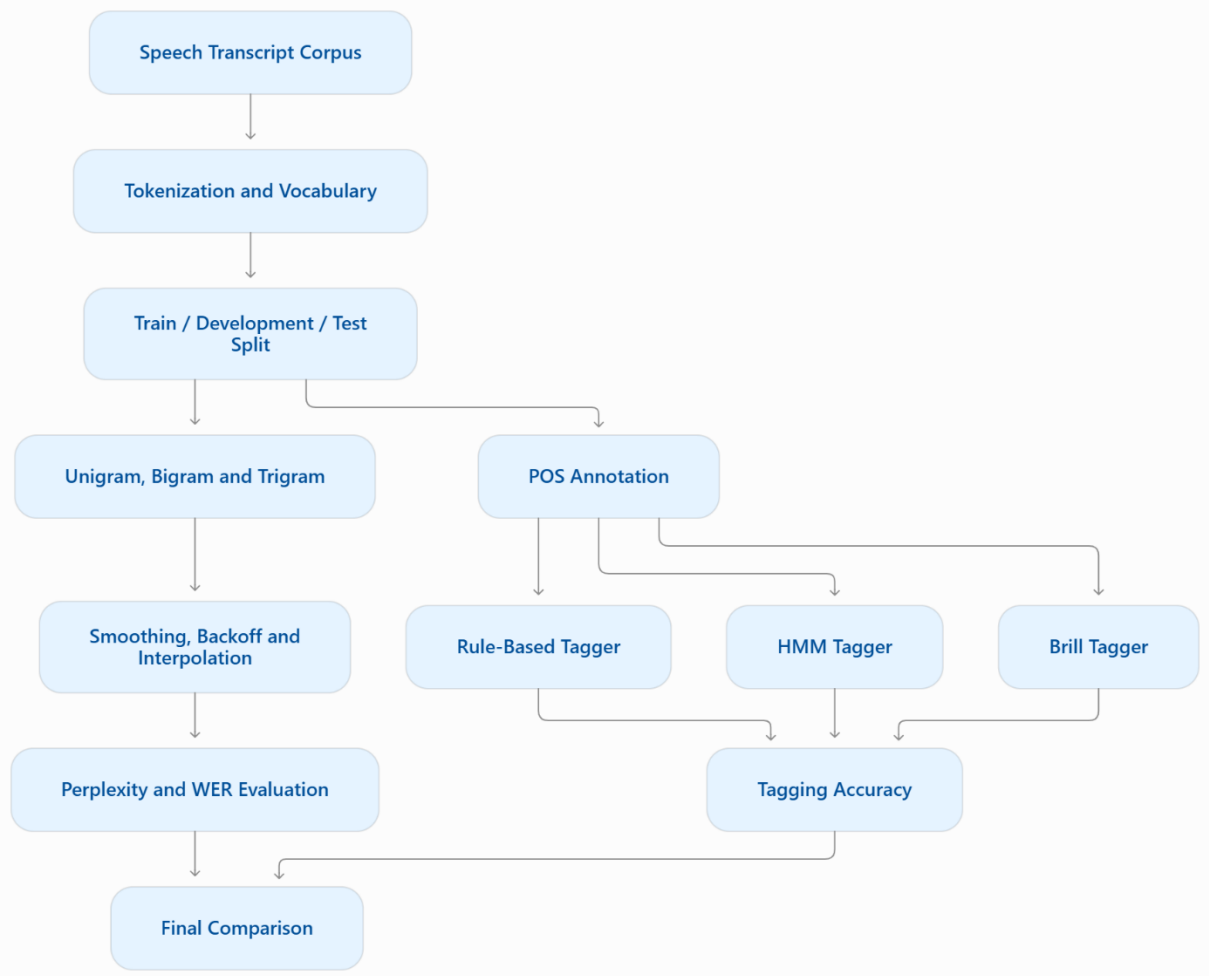
- Implement in Python the counting and vocabulary pipeline, the unigram, bigram and trigram models with every smoothing method, backoff and interpolation, and all three taggers, using the standard library and NLTK only.
- Run every model over the same splits and report, for each: training time, model size in memory, held-out perplexity or tagging accuracy, and tagging speed in tokens per second.
- Include test cases for the zero-probability utterance from Task 2, the λ weights from Task 4, the trellis from Task 8 and the first transformations from Task 9, checking each against your hand-computed result.
    - Present the results in comparable tables and at least two plots — for example perplexity against n-gram order, and tagging accuracy against training-set size.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs that match the hand calculations.

Aim

To implement and test the language models and POS taggers using Python and NLTK.



2. Implementation Requirements

Use Python standard library and NLTK to implement:

1. Token counting and vocabulary construction.
2. Unigram, bigram and trigram models.
3. Add-one and add-k smoothing.
4. Good-Turing discounting.
5. Katz backoff and interpolation.
6. Rule-based POS tagging.
7. HMM POS tagging.
8. Brill POS tagging.

3. Testing

Test	Expected behaviour
Unseen bigram	Unsmoothed model returns zero
Add-one smoothing	Unseen bigram gets non-zero probability
Interpolation	Weights sum to 1
Viterbi	Back-pointers recover best tag sequence
Brill	Learned transformations correct eligible errors
Unknown word	Unknown-word handling assigns a tag

4. Performance Table

Record the actual measurements:

Model	Training time	Memory	Perplexity / Accuracy
Unigram	Measure	Measure	Measure
Bigram	Measure	Measure	Measure
Trigram	Measure	Measure	Measure
Katz Backoff	Measure	Measure	Measure
Interpolation	Measure	Measure	Measure
Rule-based	Measure	Measure	Measure
HMM	Measure	Measure	Measure
Brill	Measure	Measure	Measure

Tagging speed:

$$ Speed=\frac{\text{Number of tokens tagged}}{\text{Time in seconds}} $$

5. Required Plots

Plot 1: Perplexity against n-gram order.

- X-axis: Unigram, bigram, trigram.
- Y-axis: Perplexity.

Plot 2: Tagging accuracy against training-set size.

- X-axis: Number of training tokens.
- Y-axis: Accuracy percentage.

Use the same test data when comparing models.

Result

The pipeline should reproduce the hand calculations and provide comparable measurements for language modelling and POS tagging.

TASK 11 – Compare, Recommend and Defend the Design 2

Marks Guiding Enquiry

Which language model and which tagger would you deploy on the helpline, and what evidence supports the choice?

Investigate and interpret:

- Compare the smoothing and backoff strategies on perplexity, word error rate after rescoring, memory and speed, and compare the three taggers on accuracy, accuracy on ambiguous and unknown tokens, speed and ease of maintenance.
- Recommend one language model and one tagger, state the expected accuracy, the failure modes, and what a caller would actually experience when the system fails.
- Discuss the manual effort involved in annotation and rule writing, and identify the point at which more annotated data would help more than more rules.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest accuracy alone is not sufficient.

Aim

To select the most suitable language model and POS tagger for the transport enquiry helpline.

1. Language Model Comparison

Model	**Strength**	**Weakness**
Unigram	Simple and fast	Ignores context
Bigram	Uses previous word	Cannot capture longer context
Trigram	Better local context	More sparsity and memory
Add-one	Easy smoothing	Excessive discounting

Add-k	Adjustable	Requires tuning
Good-Turing	Estimates unseen events	More complex
Katz Backoff	Uses shorter contexts when needed	Requires discounting and normalisation
Interpolation	Combines multiple orders	Requires weight estimation

2. POS Tagger Comparison

Tagger	**Strength**	**Weakness**
Rule-based	Easy to understand	Requires manual rules
HMM	Uses context probabilities	Unknown words are difficult
Brill	Learns readable correction rules	Requires annotated data and training

3. Recommendation

Recommended language model: Trigram model with Katz backoff.

Reason:

- Uses two previous words for context.
- Handles unseen trigrams through shorter contexts.
- Avoids the zero-probability problem.
- Suitable for phrases involving routes, destinations and ticket enquiries.

Recommended POS tagger: HMM tagger.

Reason:

- Uses contextual probabilities.
- Finds the most probable complete tag sequence.
- Viterbi provides efficient sequence decoding.
- Can be extended with unknown-word handling.

This is a theoretical recommendation. The final deployment decision must be confirmed using the measured perplexity, WER, tagging accuracy, speed and memory from Task 10.

4. Failure Modes

- An unseen station name may receive an incorrect tag.
- A new route phrase may receive a low language-model probability.
- Ambiguous words such as book may be incorrectly tagged.
- Code-mixed speech may not fit the English training vocabulary.

A caller may experience an incorrect destination, wrong route information or a request to repeat the enquiry.

5. Manual Effort

Manual work includes:

- Collecting and transcribing calls.
- Designing the tagset.
- Annotating at least 3,000 tokens.
- Writing and testing rules.
- Checking model errors.

More annotated data is preferable when the errors are caused by unseen words and varied grammatical contexts rather than missing individual rules.

Result

A trigram model with Katz backoff and an HMM tagger are reasonable starting choices. Their final selection should be based on actual experimental results, not accuracy alone.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The helpline now also answers metro and intercity rail enquiries, so hundreds of new station names, train names and fare categories enter the vocabulary every month. Callers increasingly speak code-mixed sentences, for example "Chennai to Madurai bus timing enna", and the recogniser transcribes those words phonetically and inconsistently. More than fifteen per cent of the word tokens in new calls were never seen in training, many of them exactly the proper nouns that carry the caller's intent.

Re-evaluate your original design under the changed conditions. You may: **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify what the new input breaks: a closed vocabulary, a stable unknown-word rate, English-only word classes, transition probabilities estimated on a narrower domain, and a tagset that has no category for code-mixed tokens.
- Explain why the out-of-vocabulary words are the worst possible ones to lose, given that they are mostly the place names the enquiry turns on.
- Estimate the effect on perplexity and on tagging accuracy, reasoning from the OOV rate you measured in Task 1 and the unknown-word accuracy you measured in Task 8.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended model and tagger behave on the new calls?

- Run or hand-trace at least three new utterances — one with an unseen station name, one code-mixed, one with both — through your language model and your chosen tagger, and record what fails first and why.
- Measure the degradation: held-out perplexity, word error rate after rescoring, and tagging accuracy on the new sample against your original baselines.
- Identify which component degrades fastest, and whether a different approach — a class-based or character-level model, an open-vocabulary tagger, a gazetteer of station names — would handle the new input more economically.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the models be retrained, extended or replaced?

- Choose retain, modify or redesign — for example adding a station-name gazetteer and a class-based n-gram over it, re-estimating λ weights on new data, extending the tagset with a code-mixed category, or adding a character n-gram model for unseen words.
- Describe the revised design and state exactly what changes in the vocabulary, the smoothing, the backoff weights, the tagset and the tagger.
- Determine the effect on perplexity, tagging accuracy, retraining frequency and annotation effort, and say how often the model must now be refreshed.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your perplexity tables, rescoring results, tagging accuracies, error analyses and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what an n-gram model can never capture, however much data it is given.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

A. Impact Analysis (2 Marks)

1. Changes in the Input

The original system was designed mainly for English bus and train enquiries. The new requirement introduces:

- New station and train names.
- New fare-related words.
- Code-mixed language.
- Phonetically inconsistent speech recognition.
- More than 15% out-of-vocabulary words.

2. Original Assumptions That No Longer Hold

Original assumption	**New problem**
Stable vocabulary	Vocabulary changes frequently
Low unknown-word rate	OOV rate exceeds 15%
English-only words	Code-mixed words occur
Fixed transport domain	Metro and intercity rail are added
Known station names	Many station names are unseen
Original tagset is sufficient	Code-mixed category is required

3. Why OOV Words Are Serious

Station names identify the origin and destination of a journey.

For example, in:

Chennai to Madurai bus timing enna

If the system fails to recognise Madurai, it may not identify the destination correctly.

This is more serious than losing a common word such as the or is.

4. Effect on Perplexity and Accuracy

When unknown words increase, the model has less evidence for predicting the next word.

Therefore:

- Perplexity is expected to increase.
- Language-model rescoring may become less reliable.
- POS tagging accuracy is expected to decrease.
- Destination extraction may fail.

The exact amount of degradation must be measured using the new test set.

B. Evaluation of the Original Design (2 Marks)

1. Three New Utterances

Example	**Problem**
Book ticket to Tirupati	Tirupati may be an unseen station
Chennai to Madurai bus timing enna	Contains code-mixed words
Metro train to Perambur timing sollunga	Contains unseen station and regional-language words

2. Expected Behaviour

Language model:

- May assign low probability to unseen station names.
- May incorrectly prefer familiar words.
- May fail to recognise code-mixed phrases.

HMM tagger:

- May assign unknown names to common noun tags.

- May fail to recognise regional-language words.
- May use incorrect transition probabilities.

3. Which Component Degrades First?

The vocabulary and language model may degrade quickly because unseen words directly affect word prediction.

The tagger also loses accuracy because unknown words have unreliable emissions.

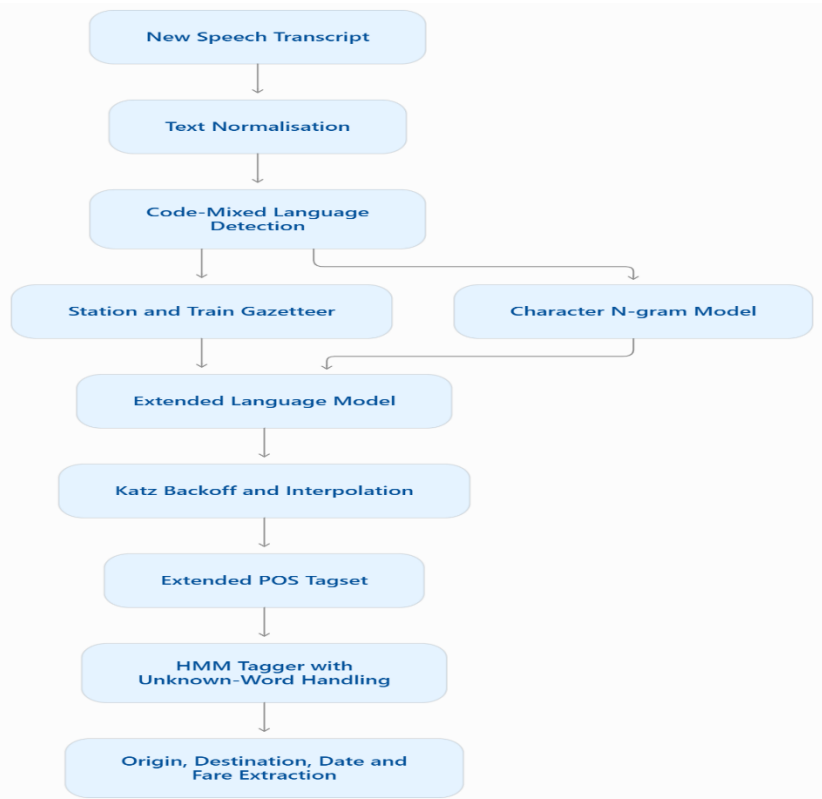
A gazetteer and character-level model can help identify unseen station names and spelling variations.

C. Adaptation or Redesign (2 Marks)

Recommended: Modify the existing design

Instead of replacing the entire system, extend it with new resources.

1. Revised Architecture



2. Changes Required

Component	Modification
Vocabulary	Add station, train and fare names
Unknown words	Improve <UNK> handling
Language model	Add character and class-based features
Smoothing	Re-estimate smoothing parameters
Backoff	Re-estimate weights using new data
Tagset	Add code-mixed and domain-specific tags
POS tagger	Add gazetteer and unknown-word features

3. Character N-gram Model

A character n-gram model uses character sequences to help recognise unfamiliar words.

For example:

Madurai → Mad, adu, dur, ura, rai

Even if a word is absent from training, its character patterns may resemble known station names.

4. Gazetteer

A gazetteer is a list of known geographical and domain-specific names.

Example:

stations = { "chennai", "madurai", "vellore", "tirupati", "perambur"}def identify_station(word): if word.lower() in stations: return "NNP" return "UNKNOWN"print(identify_station("Madurai"))

Output:

NNP

5. Retraining and Annotation

The system should be refreshed periodically.

Suggested plan:

- Add new station and train names whenever they are introduced.
- Review newly collected transcripts every week.
- Retrain the model monthly, or sooner if OOV rates or error rates increase significantly.
- Annotate additional code-mixed examples.
- Re-estimate interpolation and backoff weights using new development data.

These are proposed operational intervals, not measured requirements.

D. Evidence-Based Defence (2 Marks)

1. Why Modification Is Better

Completely replacing the original system would require more development time and resources.

Modifying the existing system allows it to retain useful English-language knowledge while supporting new vocabulary and code-mixed speech.

2. Expected Benefits

- Better recognition of new station names.
- Improved handling of spelling variations.
- Reduced unknown-word errors.
- Better POS tagging of code-mixed sentences.
- More reliable origin and destination extraction.

3. Remaining Limitations

Even after modification, the system may face:

- New words that are not in the gazetteer.
- Incorrect speech-recognition output.
- Unusual regional-language expressions.
- Ambiguous station and train names.
- Long-distance grammatical dependencies.

An n-gram model cannot fully understand unlimited long-range meaning because it considers only a fixed number of previous words. More training data helps estimate probabilities, but does not remove this structural limitation.

Final Result

The original English-focused transport helpline system should be modified rather than completely replaced. A station-name gazetteer, character n-gram model, expanded tagset, improved unknown-word handling and updated language-model weights can make the system more suitable for metro, intercity and code-mixed enquiries.

MARK DISTRIBUTION

Task	Marks
Task 1 – Counting Words and Building the Vocabulary	3
Task 2 – Unsmoothed N-gram Models	4
Task 3 – Smoothing and Discounting	5
Task 4 – Backoff and Deleted Interpolation	4
Task 5 – Entropy, Cross-Entropy and Perplexity	4
Task 6 – English Word Classes and Tagset Design	4
Task 7 – Rule-Based Part-of-Speech Tagging	4
Task 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm	5
Task 9 – Transformation-Based (Brill) Tagging	4
Task 10 – Implement and Test the Models	3
Task 11 – Compare, Recommend and Defend the Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**